

Chapter 10. ConfigMaps and Secrets

Kubernetes dedicates two primitives to defining configuration data: the *ConfigMap* and the *Secret*. Both primitives are completely decoupled from the lifecycle of a Pod, which enables you to change their configuration data values without necessarily having to redeploy the Pod.

In essence, ConfigMaps and Secrets store a set of key-value pairs. Those key-value pairs can be injected into a container as environment variables, or they can be mounted as a volume. [Figure 10-1](#) illustrates these options.

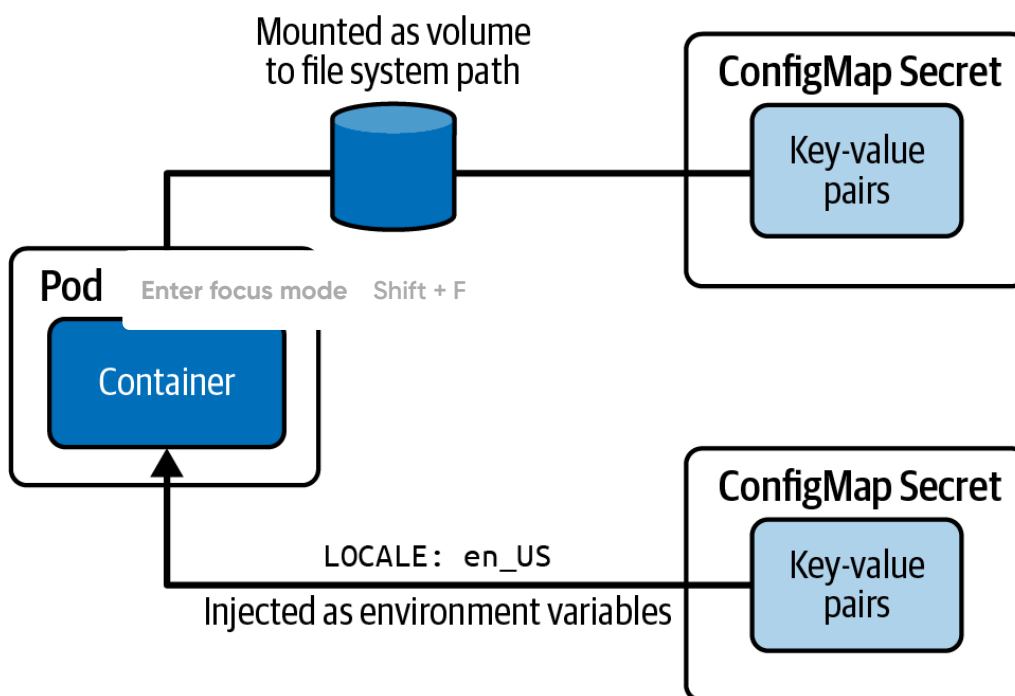


Figure 10-1. Consuming configuration data

The ConfigMap and Secret may look almost identical in purpose and structure on the surface; however, there is a slight but significant difference. A ConfigMap stores plain-text data, for example, connection URLs, runtime flags, or even structured data like a JSON or YAML content. Secrets are better suited for representing sensitive data like passwords, API keys, or Secure Sockets Layer (SSL) certificates and store the data in Base64-encoded form.

The cluster component that stores data of a ConfigMap and Secret object is etcd, which manages this data in unencrypted form by default. You can configure encryption of data in etcd, as described in the [Kubernetes documentation](#). etcd encryption is not within the scope of the exam.

This chapter references the concept of volumes heavily. Refer to [Chapter 15](#) to refresh your memory on the mechanics of consuming a volume in a Pod.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Use ConfigMaps and Secrets to configure applications
-

Working with ConfigMaps

Applications often implement logic that uses configuration data to control runtime behavior. Examples for configuration data include a connection URL and network communication options (like the number of retries or timeouts) to third-party services that differ between target deployment environments.

It's not unusual that the same configuration data needs to be made available to multiple Pods. Instead of copy-pasting the same key-value pairs across multiple Pod definitions, you can choose to centralize the information in a ConfigMap object. The ConfigMap object holds configuration data and can be consumed by as many Pods as you want. Therefore, you will need to modify the data in only one location should you need to change it.

Creating a ConfigMap

You can create a ConfigMap by emitting the imperative `create configmap` command. This command requires you to provide the source of the data as an option. Kubernetes distinguishes the four different options shown in [Table 10-1](#).

Table 10-1. Source options for data parsed by a ConfigMap

Option	Example	Description
<code>--from-literal</code>	<code>--from-literal=locale=en_US</code>	Literal values, which are key-value pairs as plain text
<code>--from-env-file</code>	<code>--from-env-file=config.env</code>	A file that contains key-value pairs and expects them to be environment variables
<code>--from-file</code>	<code>--from-file=app-config.json</code>	A file with arbitrary contents
<code>--from-file</code>	<code>--from-file=config-dir</code>	A directory with one or many files

It's easy to confuse the options `--from-env-file` and `--from-file`. The option `--from-env-file` expects a file that contains environment variables in the format `KEY=value` separated by a new line. The key-value pairs follow typical naming conventions for environment variables (e.g., the key is uppercase, and individual words are separated by an underscore character). Historically, this option has been used to process [Docker Compose .env files](#), though you can use it for any other file containing environment variables.

The `--from-env-file` option does not enforce or normalize the typical naming conventions for environment variables. The option `--from-file` points to a file or directory containing *any* arbitrary content. It's an appropriate option for files with structured configuration data to be read by an application (e.g., a properties file, a JSON file, or an XML file).

The following command shows the creation of a ConfigMap in action. We are simply providing the key-value pairs as literals:

```
$ kubectl create configmap db-config --from-literal=DB_HOST=mysql-service
--from-literal=DB_USER=backend
configmap/db-config created
```

The resulting YAML object looks like the one shown in [Example 10-1](#). As you can see, the object defines the key-value pairs in a section named `data`. A ConfigMap does not have a `spec` section.

Example 10-1. ConfigMap YAML manifest

```
apiVersion: v1
kind: ConfigMap
```

```
metadata:
  name: db-config
data:
  DB_HOST: mysql-service
  DB_USER: backend
```

You may have noticed that the key assigned to the ConfigMap data follows the typical naming conventions used by environment variables. The intention is to consume them as such in a container.

Consuming a ConfigMap as Environment Variables

With the ConfigMap created, you can now inject its key-value pairs as environment variables into a container. [Example 10-2](#) shows the use of `spec.containers[ ].envFrom[ ].configMapRef` to reference the ConfigMap by name.

Example 10-2. Injecting ConfigMap key-value pairs into the container

```
apiVersion: v1
kind: Pod
metadata:
  name: backend
spec:
  containers:
  - image: bmuschko/web-app:1.0.1
    name: backend
    envFrom:
    - configMapRef:
        name: db-config
```

After creating the Pod from the YAML manifest, you can inspect the environment variables available in the container by running the `env` Unix command:

```
$ kubectl exec backend -- env
...
DB_HOST=mysql-service
DB_USER=backend
...
```

The injected configuration data will be listed among environment variables available to the container.

Mounting a ConfigMap as a Volume

Another way to configure applications at runtime is by processing a machine-readable configuration file. Say we have decided to store the database configuration in a JSON file named *db.json* with the structure shown in [Example 10-3](#).

Example 10-3. A JSON file used for configuring database information

```
{
  "db": {
    "host": "mysql-service",
    "user": "backend"
  }
}
```

Given that we are not dealing with literal key-value pairs, we need to provide the option `--from-file` when creating the ConfigMap object:

```
$ kubectl create configmap db-config --from-file=db.json
configmap/db-config created
```

[Example 10-4](#) shows the corresponding YAML manifest of the ConfigMap. You can see that the file name becomes the key; the contents of the file have used a multiline value.

Example 10-4. ConfigMap YAML manifest defining structured data

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: db-config
data:
  db.json: |-
    {
      "db": {
        "host": "mysql-service",
        "user": "backend"
      }
    }
```

- ❶ The multiline string syntax (`|-`) used in this YAML structure removes the line feed and removes the trailing blank lines. For more information, see the [YAML syntax for multiline strings](#).

The Pod mounts the ConfigMap as a volume to a specific path inside of the container with read-only permissions. The assumption is that the application will read the configuration file when starting up. [Example 10-5](#) demonstrates the YAML definition.

Example 10-5. Mounting a ConfigMap as a volume

```
apiVersion: v1
kind: Pod
metadata:
  name: backend
spec:
  containers:
  - image: bmuschko/web-app:1.0.1
    name: backend
    volumeMounts:
    - name: db-config-volume
      mountPath: /etc/config
  volumes:
  - name: db-config-volume
    configMap:
      name: db-config
```

❶

- ❶ Assigns the volume type for referencing a ConfigMap object by name.

To verify the correct behavior, open an interactive shell to the container. As you can see in the following commands, the directory `/etc/config` contains a file with the key we used in the ConfigMap. The content represents the JSON configuration:

```
$ kubectl exec -it backend -- /bin/sh
# ls -l /etc/config
db.json
# cat /etc/config/db.json
{
  "db": {
    "host": "mysql-service",
    "user": "backend"
  }
}
```

The application code can now read the file from the mount path and configure the runtime behavior as needed.

Working with Secrets

Data stored in ConfigMaps represents arbitrary plain-text key-value pairs. In comparison to the ConfigMap, the Secret primitive is meant to represent sensitive configuration data. A typical example for Secret data is a password or an API key for authentication.

VALUES STORED IN A SECRET ARE ONLY ENCODED, NOT ENCRYPTED

Secrets expect the value of each entry to be Base64-encoded. Base64 encodes a value, but it doesn't encrypt it. Anyone with access to its value can decode it without problems. Therefore, storing Secret manifests in the source code repository alongside other resource files should be avoided.

It's somewhat unfortunate that the Kubernetes project decided to choose the term "Secret" to represent sensitive data. The nomenclature implies that data is actually secret and therefore encrypted. To keep sensitive data secure in real-world projects, you can select from a range of options.

[Bitnami Sealed Secrets](#) is a production-ready and proven Kubernetes operator that uses asymmetric crypto encryption for data. The manifest representation of the data, the CRD SealedSecret, is safe to be stored in a public source code repository. You cannot decrypt this data yourself. The controller installed with the operator is the only entity that can decrypt the data. Another option is to store sensitive data in external secrets managers, e.g., HashiCorp Vault or AWS Secrets Manager, and integrate them with Kubernetes. The [External Secrets Operator](#) synchronizes Secrets from external APIs into Kubernetes. The exam only expects you to understand the built-in Secret primitive, which is covered in the following sections.

Creating a Secret

You can create a Secret with the imperative command `create secret`. In addition, a mandatory subcommand needs to be provided that determines the type of Secret. [Table 10-2](#) lists the different types. Kubernetes assigns the value in the Internal Type column to the `type` attribute in the live object. ["Specialized Secret types"](#) discusses other Secret types and their use cases.

Table 10-2. Options for creating a Secret

CLI option	Description	Internal type
<code>generic</code>	Creates a Secret from a file, directory, or literal value	Opaque
<code>docker-registry</code>	Creates a Secret for use with a Docker registry, e.g., to pull images from a private registry when requested by a Pod	<code>kubernetes.io/dockerconfig</code>
<code>tls</code>	Creates a TLS Secret	<code>kubernetes.io/tls</code>

The most commonly used Secret type is `generic`. The options for a generic Secret are exactly the same as for a ConfigMap, as shown in [Table 10-3](#).

Table 10-3. Source options for data parsed by a Secret

Option	Example	Description
<code>--from-literal</code>	<code>--from-literal=password=secret</code>	Literal values, which are key-value pairs as plain text
<code>--from-env-file</code>	<code>--from-env-file=config.env</code>	A file that contains key-value pairs and expects them to be environment variables
<code>--from-file</code>	<code>--from-file=id_rsa</code> <code>sa=~/.ssh/id_rsa</code>	A file with arbitrary contents
<code>--from-dir</code>	<code>--from-file=config-dir</code>	A directory with one or many files

To demonstrate the functionality, let's create a Secret of type `generic`. The command sources the key-value pairs from the literals provided as a command-line option:

```
$ kubectl create secret generic db-creds --from-literal=password=s3cre!
secret/db-creds created
```

When created using the imperative command, a Secret will automatically Base64-encode the provided value. This can be observed by looking at the produced YAML manifest. You can see in [Example 10-6](#) that the value `s3cre!` has been turned into `czNjcMUm`, the Base64-encoded equivalent.

Example 10-6. A Secret with Base64-encoded values

```
apiVersion: v1
kind: Secret
metadata:
  name: db-creds
type: Opaque ❶
data:
  pwd: czNjcmUh ❷
```

- ❶ The value `Opaque` for the `type` has been assigned to represent generic sensitive data.
- ❷ The plain-text value has been Base64-encoded automatically if the object has been created imperatively.

If you start with the YAML manifest to create the Secret object, you will need to create the Base64-encoded value if you want to assign it to the `data` attribute. A Unix tool that does the job is `base64`. The following command achieves exactly that:

```
$ echo -n 's3cre!' | base64
czNjcmUh
```

As a reminder, if you have access to a Secret object or its YAML manifest, then you can decode the Base64-encoded value at any time with the `base64` Unix tool. Therefore, you may as well specify the value in plain text when defining the manifest, which is discussed in the next section.

Defining Secret data with plain-text values

Having to generate and assign Base64-encoded values to Secret manifests can become cumbersome. The Secret primitive offers the `stringData` attribute as a replacement for the `data` attribute. With `stringData`, you can assign plain-text values in the manifest file, as shown in

[Example 10-7](#).

Example 10-7. A Secret with plain-text values

```
apiVersion: v1
kind: Secret
metadata:
  name: db-creds
type: Opaque
```

```
stringData: ❶  
  pwd: s3cre! ❷
```

- ❶ The `stringData` attribute allows assigning plain-text key-value pairs.
- ❷ The value referenced by the `pwd` key was provided in plain-text format.

Kubernetes will automatically Base64-encode the `s3cre!` value upon creation of the object from the manifest. The result is the live object representation shown in [Example 10-8](#), which you can retrieve with the command `kubectl get secret db-creds -o yaml`.

Example 10-8. A Secret live object

```
apiVersion: v1  
kind: Secret  
metadata:  
  name: db-creds  
type: Opaque  
data: ❶  
  pwd: czNjcmUh ❷
```

- ❶ The live object of a Secret always uses the `data` attribute even though you may have used `stringData` in the manifest.
- ❷ The value has been Base64-encoded upon creation.

You can represent arbitrary Secret data using the `Opaque` type. Kubernetes offers specialized Secret types you can choose from should the data fit specific use cases. The next section discusses those specialized Secret types.

Specialized Secret types

Instead of using the `Opaque` Secret type, you can also use one of the [specialized types](#) to represent configuration data for particular use cases. The type `kubernetes.io/basic-auth` is meant for basic authentication and expects the keys `username` and `password`. At the time of writing, Kubernetes does not validate the correctness of the assigned keys.

The created object from this definition automatically Base64-encodes the values for both keys. [Example 10-9](#) illustrates a YAML manifest for a

Secret with type `kubernetes.io/basic-auth`.

Example 10-9. Usage of the Secret type `kubernetes.io/basic-auth`

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-basic-auth
type: kubernetes.io/basic-auth
stringData: ❶
  username: bmuschko ❷
  password: secret ❷
```

- ❶ Uses the `stringData` attribute to allow for assigning plain-text values
- ❷ Specifies the mandatory keys required by the `kubernetes.io/basic-auth` Secret type

Consuming a Secret as Environment Variables

Consuming a Secret as environment variables works similar to the way you'd do it for ConfigMaps. Here, you'd use the YAML expression `spec.containers[ ].envFrom[ ].secretRef` to reference the name of the Secret. [Example 10-10](#) injects the Secret named `secret-basic-auth` as environment variables into the container named `backend`.

Example 10-10. Injecting Secret key-value pairs into the container

```
apiVersion: v1
kind: Pod
metadata:
  name: backend
spec:
  containers:
    - image: bmuschko/web-app:1.0.1
      name: backend
      envFrom:
        - secretRef:
            name: secret-basic-auth
```

Inspecting the environment variables in the container reveals that the Secret values do not have to be decoded. That's something Kubernetes does automatically. Therefore, the running application doesn't need to implement custom logic to decode the value. Note that Kubernetes does

not verify or normalize the typical naming conventions of environment variables, as you can see in the following output:

```
$ kubectl exec backend -- env
...
username=bmuschko
password=secret
...
```

Remapping Environment Variable Keys

Sometimes, key-value pairs stored in a Secret do not conform to typical naming conventions for environment variables or can't be changed without impacting running services. You can redefine the keys used to inject an environment variable into a Pod with the

`spec.containers[ ].env[ ].valueFrom` attribute. [Example 10-11](#) turns the key `username` into `USER` and the key `password` into `PWD`.

Example 10-11. Remapping environment variable keys for Secret entries

```
apiVersion: v1
kind: Pod
metadata:
  name: backend
spec:
  containers:
  - image: bmuschko/web-app:1.0.1
    name: backend
    env:
    - name: USER
      valueFrom:
        secretKeyRef:
          name: secret-basic-auth
          key: username
    - name: PWD
      valueFrom:
        secretKeyRef:
          name: secret-basic-auth
          key: password
```

The resulting environment variables available to the container now follow the typical conventions for environment variables, and we changed how they are consumed by the application code:

```
$ kubectl exec backend -- env
...
USER=bmuschko
PWD=secret
...
```

The same mechanism of reassigning environment variables works for ConfigMaps. You'd use the attribute `spec.containers[ ].env[ ].valueFrom.configMapRef` instead.

Mounting a Secret as a Volume

To demonstrate mounting a Secret as a volume, we'll create a new Secret of type `kubernetes.io/ssh-auth`. This Secret type captures the value of an SSH private key that you can view using the command `cat ~/.ssh/id_rsa`. To process the SSH private key file with the `create secret` command, it needs to be available as a file with the name `ssh-privatekey`:

```
$ cp ~/.ssh/id_rsa ssh-privatekey
$ kubectl create secret generic secret-ssh-auth --from-file=ssh-privatekey
--type=kubernetes.io/ssh-auth
secret/secret-ssh-auth created
```

Mounting the Secret as a volume follows the two-step approach: define the volume first, and then reference it as a mount path for one or many containers. The volume type is called `secret`, as used in [Example 10-12](#).

Example 10-12. Mounting a Secret as a volume

```
apiVersion: v1
kind: Pod
metadata:
  name: backend
spec:
  containers:
    - image: bmuschko/web-app:1.0.1
      name: backend
      volumeMounts:
        - name: ssh-volume
          mountPath: /var/app
          readOnly: true
  volumes:
    - name: ssh-volume
```

```
secret:
```

```
secretName: secret-ssh-auth
```

②

- ① Files provided by the Secret mounted as volume cannot be modified.
- ② Note that the attribute `secretName` that points to the Secret name is not the same as for the ConfigMap (which is `name`).

You will find the file named `ssh-privatekey` in the mount path `/var/app`. To verify, open an interactive shell and render the file contents. The contents of the file are not Base64-encoded:

```
$ kubectl exec -it backend -- /bin/sh
# ls -l /var/app
ssh-privatekey
# cat /var/app/ssh-privatekey
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: AES-128-CBC,8734C9153079F2E8497C8075289EBBF1
...
-----END RSA PRIVATE KEY-----
```

Summary

Application runtime behavior can be controlled either by injecting configuration data as environment variables or by mounting a volume to a path. In Kubernetes, this configuration data is represented by the API resources ConfigMap and Secret in the form of key-value pairs. A ConfigMap is meant for plain-text data, and a Secret encodes the values in Base64 to obfuscate the values. Secrets are a better fit for sensitive information like credentials and SSH private keys.

Exam Essentials

Practice creating ConfigMap objects with the imperative and declarative approaches

The quickest ways to create those objects are the imperative `kubectl create configmap` commands. Understand how to provide the data with the help of different command-line flags. The ConfigMap specifies plain-text key-value pairs in the `data` section of YAML manifest.

Practice creating Secret objects with the imperative and declarative approaches

Creating a Secret using the imperative command `kubectl create secret` does not require you to Base64-encode the provided values. `kubectl` performs the encoding operation automatically. The declarative approach requires the Secret YAML manifest to specify a Base64-encoded value with the `data` section. You can use the `stringData` convenience attribute in place of the `data` attribute if you prefer providing a plain-text value. The live object will use a Base64-encoded value. Functionally, there's no difference at run-time between the use of `data` and `stringData`.

Understand the purpose of specialized Secret types

Secrets offer specialized types, e.g., `kubernetes.io/basic-auth` or `kubernetes.io/service-account-token`, to represent data for specific use cases. Read up on the different types in the Kubernetes documentation, and understand their purpose.

Know how to inspect ConfigMap and Secret data

The exam may confront you with existing ConfigMap and Secret objects. You need to understand how to use the `kubectl get` or the `kubectl describe` command to inspect the data of those objects. The live object of a Secret will always represent the value in a Base64-encoded format.

Exercise the consumption of ConfigMaps and Secrets in Pods

The primary use case for ConfigMaps and Secrets is the consumption of the data from a Pod. Pods can inject configuration data into a container as environment variables or mount the configuration data as volumes. For the exam, you need to be familiar with both consumption methods.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. In this exercise, you will first create a ConfigMap from a YAML configuration file as a source. Later, you'll create a Pod, consume the ConfigMap as volume, and inspect the key-value pairs as files. Navigate to the directory `app-a/ch10/configmap` of the checked-out GitHub repository [bmuschko/cka-study-guide](#). Inspect the YAML configuration file named `application.yaml`.

Create a new ConfigMap named `app-config` from that file.

Create a Pod named `backend` that consumes the ConfigMap as volume at the mount path `/etc/config`. The container runs the image `nginx:1.23.4-alpine`.

Shell into the Pod and inspect the file at the mounted volume path.

2. You will first create a Secret from literal values in this exercise. Next, you'll create a Pod and consume the Secret as environment variables. Finally, you'll print out its values from within the container.

Create a new Secret named `db-credentials` with the key-value pair `db-password=password`.

Create a Pod named `backend` that uses the Secret as an environment variable named `DB_PASSWORD` and runs the container with the image `nginx:1.23.4-alpine`.

Shell into the Pod and print out the created environment variables.

You should be able to find the `DB_PASSWORD` variable.